MARRAKECH
جامعة القاضي عياض
UNIVERSITÉ CADI AYYAD

Semlalia
Faculté des Sciences Semlalia
Département de L'informatique

python for AI

# Python pour l'Intelligence Artificielle
# Chapitre 2 : Les calculs scientifiques

Licence IA SD - S5

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Qu'est-ce que NumPy ?

- Le terme NumPy est en fait l'abréviation de « Numerical Python ». Il s'agit d'une bibliothèque gratuite / Open Source en langage Python.
- écrit en C et en Fortran → performances élevées pour calculs vectorisés ( opérations appliquées sur des tableaux entiers (vecteurs, matrices, n-dimensions) sans utiliser de boucle explicite (for/while) ).
- NumPy (Numerical Python) est une bibliothèque Python spécialisée dans :
  - Le calcul scientifique.
  - La manipulation efficace de tableaux de données (appelés `ndarray`).
  - Les opérations mathématiques vectorisées.
  - Le calcul matriciel, les fonctions statistiques et les algèbres linéaires.

NumPy

http://www.numpy.org

- Pourquoi NumPy en IA ?
  - NumPy est la base de nombreuses bibliothèques comme **Pandas**, **SciPy**, **Scikit-learn**.
  - Il permet de traiter rapidement de grandes quantités de données numériques.
  - Il optimise les performances grâce à des opérations vectorisées (plus rapides que les boucles Python classiques).
- On utilise cet outil pour la programmation scientifique en Python, et notamment pour la programmation en Data Science, pour l'ingénierie, les mathématiques ou la science.
- NumPy est une bibliothèque Python qui permet à la fois de manipuler des matrices (des tableaux multidimensionnels) mais aussi de faire des calculs sur ces objets.

# Présentation de NumPy

- La librairie numpy est consacrée entièrement au calcul numérique en Python. Elle utilise essentiellement des variables de type `ndarray` (en abrégé array), qui l'on peut voir comme des tableaux à plusieurs dimensions.
- Il y a des différences majeures avec les listes (resp. les listes de listes) qui pourraient elles aussi nous servir à représenter des vecteurs (resp. des matrices) :
  - – Les tableaux numpy sont homogènes, c'est-à-dire constitués d'éléments du même type. On trouvera donc des tableaux d'entiers, des tableaux de flottants, des tableaux de chaînes de caractères, etc.
  - – La taille des tableaux numpy est fixée à la création. On ne peut donc augmenter ou diminuer la taille d'un tableau comme le ferait pour une liste (à moins de créer un tout nouveau tableau, bien sûr).
- Ces contraintes sont en fait des avantages :
  - – Le format d'un tableau numpy et la taille des objets qui le composent étant fixés, l'empreinte du tableau en mémoire est invariable et l'accès à ses éléments se fait en temps constant.
  - – Les opérations sur les tableaux sont optimisées en fonction du type des éléments, et sont beaucoup plus rapides qu'elles ne le seraient sur des listes équivalentes.

# ndarray vs list

- Les calculs avec numpy sont particulièrement optimisés car les array sont homogènes (ils ne contiennent que des valeurs d'un même type) et de taille fixée à la création. # `list`
- NumPy ne parcourt pas les éléments un par un en Python. Il délègue la boucle à du code bas niveau (C), optimisé pour le calcul parallèle et la gestion mémoire.Cela rend l'opération des centaines de fois plus rapide qu'une boucle Python

```
import numpy as np
L = list(range(1000))
a = np.array(L)

# python classique avec compréhension
%timeit [x**2 for x in L ]
# "vectorisation" de l'array numpy (opérateur **)
%timeit a**2

# python avec un array numpy
%timeit [x**2 for x in a ]
```

c'est 300 fois plus rapide.

on perd complètement les avantages de numpy !
Quand on boucle, on traite les éléments un par un avec du python basique. Donc:

**Ne jamais boucler sur un tableau numpy**


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# NumPy

NumPy repose sur **quatre** concepts essentiels qui structurent toute son architecture et permettent de réaliser des calculs scientifiques rapides et efficaces.

- **Ndarray (N-dimensional array) :** Tableau multidimensionnel homogène ; `a = np.array([[1,2,3],[4,5,6]])`
- **Indexation avancée :** Sélection et filtrage puissants ; `a[a>10], a[[1,3]]`
- **Vectorisation :** Calcul rapide sans boucle ; `a + b, a**2`
- **Broadcasting :** Opérations entre tailles différentes ; `a + b` avec b plus petit

# Présentation de NumPy

- Pour l'importer, on recommande d'utiliser :

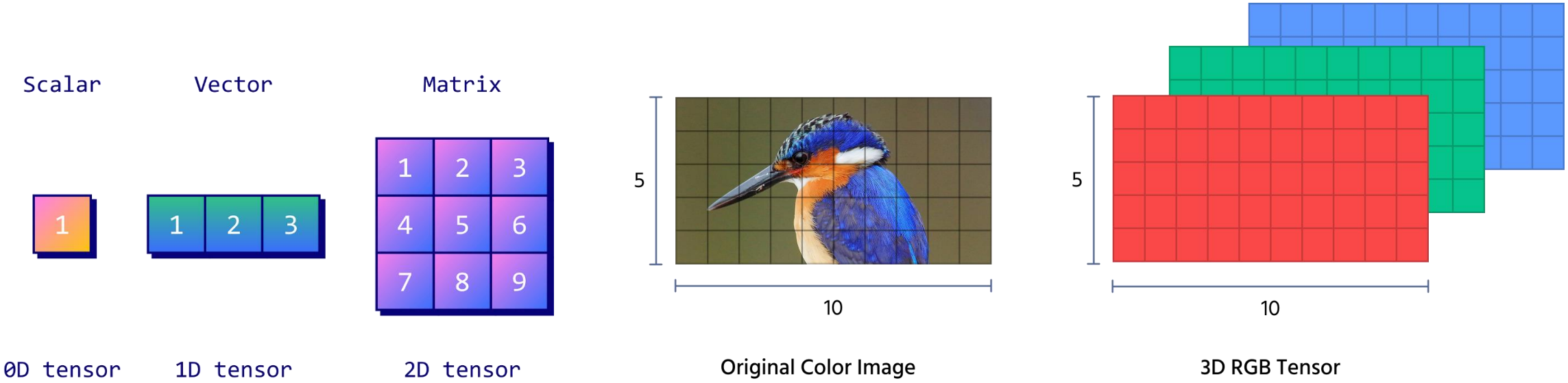
```
import numpy as np
```

- Toutes les **fonctions** NumPy seront alors préfixées par `np`. Par contre, on rappelle que pour une **méthode** il n'y a pas de préfixe à utiliser.
  - Une **fonction** est une **instruction indépendante** définie globalement (souvent dans un module, comme NumPy) ( `y = np.sqrt(x)` )
  - Une **méthode** est une **fonction "attachée" à un objet**. ( `x.mean()` )


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# array/arrays

- Dans la terminologie numpy : **vecteurs**, **matrices** et autres **tenseurs** sont appelés arrays ( tableaux).
  - **Scalaire (tenseur 0D)** : un seul nombre, comme `5` ou `3.14`;
  - **Vecteur (tenseur 1D)** : une liste de nombres, telle que `[1, 2, 3]`;
  - **Matrice (tenseur 2D)** : une grille 2D de nombres, comme un tableau avec des lignes et des colonnes.
  - **Tenseurs de dimensions supérieures** (3D, 4D, etc.) étendent le concept de **matrices** à des dimensions supplémentaires. Par exemple, un tenseur 3D peut représenter une image avec hauteur, largeur et canaux de couleur.



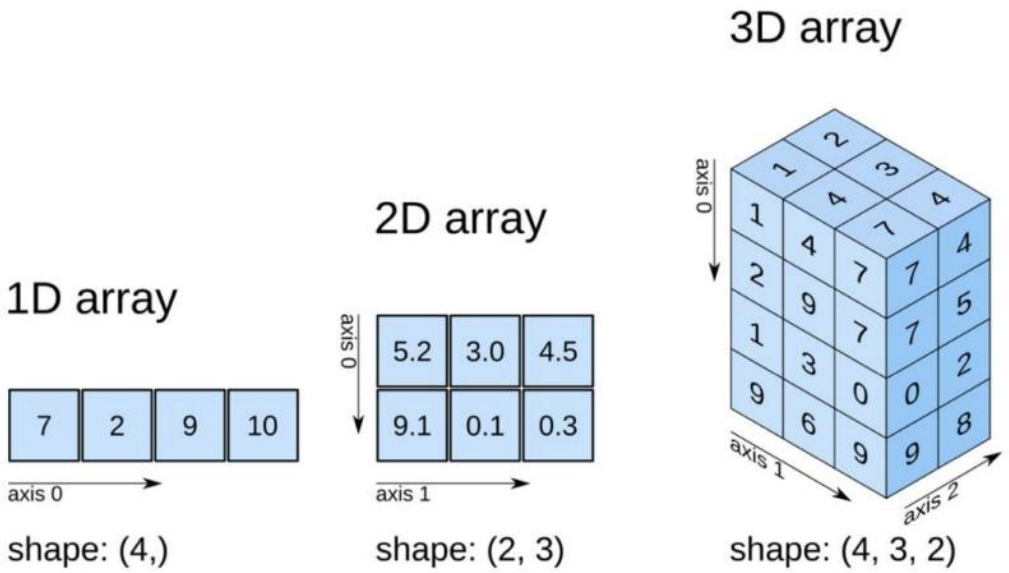


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Le type ndarray

- Un tableau numpy est de type `ndarray`
- On parle de tableau numpy ou de `ndarray` (plutôt tableau numpy).
- C'est une zone de mémoire contigüe qui en contient que des éléments du même type.
  - Comme la mémoire est contigüe, l'accès aux données peut être très rapide.
  - Pour chaque type de donnée, on aura des fonctions spécialisées.
- On peut changer les valeurs d'un tableau, à condition de garder le même type qu'on appelle « `dtype` » (data type : un type python ou un type équivalent numpy. Par exemple : int64).
  - **En python**, un tableau est une liste qui peut contenir n'importe quel type de données.
  - **En numpy**, c'est un tableau d'un type fixe. → Numpy interprète les chaines correspondant à un nombre comme un nombre si nécessaire et réciproquement !
- On ne peut pas changer la taille d'un tableau : ajouter ou supprimer des éléments. La taille du tableau numpy est fixe.
  - En python, un tableau est une liste dans laquelle on peut ajouter de nouveaux éléments.
  - En numpy, la taille est fixe.

# Le type ndarray




| | | **Vecteur** | **Matrice** | **Pavé** |
|---|---|---|---|---|
| Type | dtype | int64 | float64 | int64 |
| Dimension | ndim | 1 | 2 | 3 |
| Format | shape | (4, ) | (2, 3) | (4, 3, 2) |
| Nombre d'éléments | size | 4 | 6 | 24 |
| Taille en octets | nbytes | 32 | 48 | 192 |
| Taille d'un élément en octet | itemsize | 8 | 8 | 8 |

# Fonction array et « data types »



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Création de tableaux avec **array**

- On utilise en général la fonction `array` pour former un tableau à partir de la liste (ou la liste des listes) de ses éléments.
- La fonction `array` agit comme un mécanisme de conversion vers `numpy.ndarray` qui est le type commun à tous les tableaux numpy.
- On ne confondra pas avec le résultat renvoyé par l'attribut `dtype` (abréviation de *data type*) de `a`, et qui indique le type commun à tous les éléments de celui-ci (en l'occurrence des entiers codés sur 32 bits, c'est-à-dire quatre octets ou *bytes*).

```
>>> import numpy as np
>>> L = list(range(1000))
>>> a = np.array(L)
>>> b = np.array([1,2, 3, 4, 5])
>>> c = np.array([[1,2, 3], [4, 5, 6]])
>>> type(a)
<class 'numpy.ndarray'>
>>> a.dtype
dtype('int32')
```

```
tab=np.array( list(range(5)) )
tab=np.array( range(5) )
tab=np.array( [ i for i in range(5) ] )
tab=np.array( [ i%2 for i in range(5) ] )
```

# Création de tableaux avec **array**

- la *méthode* `tolist` d'un tableau numpy le transforme en la liste (éventuellement une liste de listes) de ses éléments.
- Attention, ce n'est pas une *fonction* du module numpy. On n'écrira donc pas `np.tolist(a)` mais `a.tolist()`

```
>>> a
array([[ 0, 1, 2, 3, 4],
       [ 5, 6, 7, 8, 9],
       [10, 11, 12, 13, 14]])
>>> b = a.tolist(); b # convertit a en une liste de listes
[[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]
```

# Fonction array()

- La fonction array transforme une liste pythone (ou la liste des listes) en tableau numpy
- On renomme numpy en « np » à l'importation : c'est l'usage.
- On ne peut pas ajouter ou supprimer un élément dans un tableau numpy.
- Les éléments ont tous le même type qu'on appelle dtype.

```
import numpy as np
L = list(range(1000))
a = np.array(L)
b = np.array([1,2, 3, 4, 5])
c = np.array([[1,2, 3], [4, 5, 6]])
```

```
tab=np.array( list(range(5)) )
tab=np.array( range(5) )
tab=np.array( [ i for i in range(5) ] )
tab=np.array( [ i%2 for i in range(5) ] )
```

# Fonction array()

- La fonction array transforme une liste pythone (ou la liste des listes) en tableau numpy
- On renomme numpy en « np » à l'importation : c'est l'usage.
- On ne peut pas ajouter ou supprimer un élément dans un tableau numpy.
- Les éléments ont tous le même type qu'on appelle dtype.

```
import numpy as np
L = list(range(1000))
a = np.array(L)
b = np.array([1,2, 3, 4, 5])
c = np.array([[1,2, 3], [4, 5, 6]])
```

```
tab=np.array( list(range(5)) )
tab=np.array( range(5) )
tab=np.array( [ i for i in range(5) ] )
tab=np.array( [ i%2 for i in range(5) ] )
```

# Création d'array par fonction de génération

- Deux fonctions essentielles pour générer des suites de nombres sont :
  - `np.linspace(start, stop, num) :` permet de créer un tableau 1d dont les n composantes sont uniformément réparties entre a et b (attention, ici b est inclus)
  - `np.arange(start, stop, step) :` permet de créer un tableau 1d et fonctionne comme la fonction range (a,b,inc) sauf que les arguments peuvent être des flottants (attention, comme avec range, la valeur finale est exclue) :

- Elles servent souvent à :
  - créer des vecteurs d'entrée pour les fonctions mathématiques,
  - générer des plages de valeurs pour des tests,

**start :** Valeur de départ

**stop :** Valeur de fin

**num :** Nombre de points générés

**step :** Pas d'incrémentation

# Création d'array par fonction de génération

- les fonctions « zéros » et « ones » permettent de créer des tableaux contenants des 0 ou des 1.
- Attention pour créer des tableaux 2d et plus, les dimensions doivent être données sous la forme d'un seul argument tuple (contenant ces dimensions). Quelques exemples :

```
>>> np.zeros(5) # tableau 1d de 0 : array([ 0., 0., 0., 0., 0.])
>>> np.ones((2,4)) # tableau 2d de taille 2 x 4 contenant des 1
# array([[ 1., 1., 1., 1.],
#        [ 1., 1., 1., 1.]])
```

```
>>> np.zeros((2,2,2)) # tableau 3d de taille 2 x 2 x 2 contenant des 0
# array([[[ 0., 0.],[ 0., 0.]],
#       [[ 0., 0.], [ 0., 0.]]])
```

```
>>> np.zeros((3,), dtype=int)
array([0, 0, 0])
```

# Création d'array par fonction de génération

- `np.full()` permet de créer un tableau (ou une matrice) rempli d'une valeur constante.

  `np.full(shape, value)`
- `np.eye()` crée une matrice identité. C'est une matrice carrée avec :
  - des 1 sur la diagonale principale,
  - des 0 partout ailleur

  `np.eye(N, M=None, k=0)`
    - N : Nombre de lignes
    - M : Nombre de colonnes (optionnel, = N si non précisé)
    - k : Décalage de la diagonale (0 = principale, +1 = au-dessus, -1 = en dessous

```
v = np.full(5, 7)
# [7 7 7 7 7]
m = np.full((3, 4), 2)
#[[2 2 2 2]
# [2 2 2 2]
# [2 2 2 2]]
```

```
I = np.eye(3)
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]
A = np.eye(4, 5, k=1)
# [[0. 1. 0. 0. 0.]
#  [0. 0. 1. 0. 0.]
#  [0. 0. 0. 1. 0.]
#  [0. 0. 0. 0. 1.]]
```

# Attributs de dimension d'un tableau

- `shape` indique le format du tableau, sous la forme du tuple du nombre d'éléments dans chaque direction : ici le couple (3, 4) indique qu'il y a trois lignes et quatre colonnes.
- la fonction `alen` (ce n'est pas un *attribut*) donne la première dimension d'un tableau (la taille pour un vecteur, le nombre de lignes pour une matrice).
- `size` donne le nombre total d'éléments,

  variante : `np.size(a,0)` et `np.size(a,1)` donnent respectivement le nombre de lignes et le nombre de colonnes d'une matrice.
- `ndim` renvoie le nombre d'indices nécessaires au parcours du tableau (usuellement : 1 pour un vecteur, 2 pour une matrice).

```
>>> a.shape
(3, 4)
>>> np.alen(a)
3
>>> a.size
12
>>> np.size(a,0)
3
>>> np.size(a,1)
4
>>> a.ndim
2
```

Les fonctions `shape`, `size`, et `ndim` peuvent être évoquées à la fois comme des attributs d'un tableau et comme des fonctions du module numpy prenant un tableau en argument. Pour prendre l'exemple de shape, on peut donc écrire aussi bien « `a.shape` » que « `np.shape(a)` ».

# Le « data type » d'un tableau

- Le **data type** est automatiquement choisi par la fonction array au vu des coefficients.
- En modifiant à peine l'exemple précédent, il suffit d'un seul flottant (on a changé 4 en 4.0) pour que le tableau tout entier soit considéré comme formé de flottants (ici codés sur 64 bits).

```
>>> b = np.array([[8,3,2,4],[5,1,6,0],[9,7,4.0,1]])
>>> b
array([[ 8., 3., 2., 4.],
        [ 5., 1., 6., 0.],
        [ 9., 7., 4., 1.]])
>>> b.dtype
dtype('float64')
```

- Il est toujours possible de forcer le data type du tableau avec l'option `dtype` de la fonction array (à condition de faire du upcasting, c'est-à-dire de forcer le typage « vers le haut », basé sur les inclusions strictes $\mathbb{Z} \subset \mathbb{R} \subset \mathbb{C}$

```
>>> c = np.array([[8,3,2,4],[5,1,6,0],[9,7,4,1]],
dtype=complex)
>>> c
array([[ 8.+0.j, 3.+0.j, 2.+0.j, 4.+0.j],
        [ 5.+0.j, 1.+0.j, 6.+0.j, 0.+0.j],
        [ 9.+0.j, 7.+0.j, 4.+0.j, 1.+0.j]])
>>> c.dtype
dtype('complex128')
```

# Copie d'un tableau avec conversion de « data type »

- On peut créer une copie d'un tableau d'un data type à un autre en utilisant sa méthode `astype` .
- Pour illustrer ces possibilités on reprend les définitions précédentes de `a (dtype=int), b (dtype=float) et c (dtype=complex).`
- On retiendra que la méthode `astype` crée une copie du tableau (donc n'affecte pas le contenu de la variable).

```
>>> a.astype(float)
array([[ 0., 1., 2., 3., 4.],
       [ 5., 6., 7., 8., 9.],
       [10., 11., 12., 13., 14.]])
```

effectue une copie du tableau *a* en convertissant vers le type float

```
>>> b.astype(int)
array([[8, 3, 2, 4],
       [5, 1, 6, 0],
       [9, 7, 4, 1]])
```

effectue une copie du tableau *b* en convertissant vers le type int

```
>>> c.astype(float)
array([[ 8., 3., 2., 4.],
       [ 5., 1., 6., 0.],
       [ 9., 7., 4., 1.]])
```

effectue une copie du tableau *c* en convertissant vers le type float

- Un tableau numpy occupe une place constante en mémoire, égale au produit du nombre de ses éléments par la taille (fixe) d'un élément du data type (en principe : 4 octets pour un entier, 8 pour un flottant, 16 pour un nombre complexe).

# Lecture et écriture dans un tableau


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Lecture de valeurs dans un vecteur, « slicing »

- La lecture d'éléments d'un tableau numpy procède par coupes (« slices » en anglais) suivant une ou éventuellement plusieurs des dimensions du tableau.
- Une coupe est un mécanisme de sélection de positions régulièrement espacées dans un intervalle `I = [0, p - 1]` .
- La spécification la plus générale d'une coupe est :

  `début(inclus) : fin(exclue) : incrément`

- Cela désigne les positions de *I* à partir de la valeur de début (incluse), par incrémentations (**s**tep) successives tant qu'on reste dans l'intervalle défini par les valeurs de début (**s**tart) et de fin (**s**top) (cette dernière étant exclue : très important).
- Les positions acceptées par l'interpréteur Python forment en fait l'intervalle `I' = [-p, . . . , -2, -1, 0, 1, 2,...,p-1]` et elles sont converties « modulo p » en une position effective dans `I = [0, p - 1]`.
- L'intérêt de cette extension syntaxique est que les positions négatives successives `-1, -2,...,` qui sont converties en les positions `p - 1, p - 2,...` (où *p* représente la longueur du vecteur *v*) permettent de lire les éléments de *v* à partir de la fin.

# Lecture de valeurs dans un vecteur, « slicing »

- Lire des éléments individuels dans le vecteur *v*.

```
>>> v
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150])
```

```
>>> v[0] # le 1er élément
0
>>> v[1] # le 2ième élément
10
>>> v[2] # le 3ième élément
20
```

ici on compte à partir du début

```
>>> v[v.size-1] # le dernier
150
>>> v[-16] # le premier
0
>>> v[-v.size] # le premier
0
```

ici on compte à partir de la fin

```
>>> v[-1] # le dernier élément
150
>>> v[-2] # l'avant-dernier
140
>>> v[-3] # l'avant-avant-dernier
130
```

ici on compte à partir de la fin

# Lecture de valeurs dans un vecteur, « slicing »

- On continue, avec le même vecteur v, en effectuant quelques coupes de valeurs consécutives dans l'ordre des indices croissants, c'est-à-dire avec la valeur 1 d'incrémentation par défaut :

```
>>> v
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150])
```

```
>>> v[4:11] # de v[4] à v[10], donc 11-4 = 7 éléments consécutifs
array([ 40, 50, 60, 70, 80, 90, 100])
>>> v[:6] # de v[0] à v[5], les 6 premières valeurs de v
array([ 0, 10, 20, 30, 40, 50])
>>> v[6:] # (v étant de longueur p=16), de v[6] à v[p-1], les p-6 = 10 dernières valeurs de v
array([ 60, 70, 80, 90, 100, 110, 120, 130, 140, 150])
>>> v[:] # la totalité du vecteur v, comme si on avait écrit v[0:p]
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150])
```

# Lecture de valeurs dans un vecteur, « slicing »

- Voici maintenant des coupes d'éléments consécutifs, mais à l'envers, et toujours avec le même vecteur **v** :

```
>>> v
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150])
```

```
>>> v[11:4:-1] # de v[11] à v[5], donc |4-11| = 7 éléments
array([110, 100, 90, 80, 70, 60, 50])
>>> v[:6:-1] # de v[p-1=15] à v[7], donc |6-15| = 9 éléments
array([150, 140, 130, 120, 110, 100, 90, 80, 70])
>>> v[6::-1] # de v[6] à v[0], donc 7 éléments
array([60, 50, 40, 30, 20, 10, 0])
>>> v[::-1] # la totalité du vecteur, mais à l'envers
array([150, 140, 130, 120, 110, 100, 90, 80, 70, 60, 50, 40, 30, 20, 10, 0])
```

# Lecture de valeurs dans un vecteur, « slicing »

- coupes du même vecteur ***v***, mais avec un incrément différent de **1** ou **−1** :

```
>>> v
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150])
```

```
>>> v[::4]
array([ 0, 40, 80, 120])
>>> v[1::4]
array([ 10, 50, 90, 130])
>>> v[::5]
array([ 0, 50, 100, 150])
```

```
>>> v[:6:2]
array([ 0, 20, 40])
>>> v[:7:2]
array([ 0, 20, 40, 60])
>>> v[10::2]
array([100, 120, 140])
```

```
>>> v[:5:-3]
array([150, 120, 90, 60])
>>> v[:6:-3]
array([150, 120, 90])
>>> v[6::-3]
array([60, 30, 0])
```

Notez que lorsqu'aucun indice n'est indiqué à début (start) ou à fin (stop) , Python prend par défaut tous les éléments depuis le début ou tous les éléments jusqu'à la fin respectivement.

# Lecture de valeurs dans une matrice

- On procède comme dans la sous-section précédente, en effectuant une coupe suivant la première et/ou suivant la deuxième dimension (ça se généralise bien sûr à des tableaux de trois dimensions ou plus).
- On se contentera de quelques exemples avec cette matrice `m` d'ordre `5 × 8`, de terme général `m[i, j] = 10i+j` (en numérotant lignes et colonnes à partir de 0) :

```
>>> m = np.array([[10*i+j for j in range(8)] for i in range(5)])
>>> m
array([[ 0, 1, 2, 3, 4, 5, 6, 7],
        [10, 11, 12, 13, 14, 15, 16, 17],
        [20, 21, 22, 23, 24, 25, 26, 27],
        [30, 31, 32, 33, 34, 35, 36, 37],
        [40, 41, 42, 43, 44, 45, 46, 47]])
```

Rappel : Une **compréhension de liste** est une manière **courte et claire** de créer une liste en une seule ligne, sans utiliser de boucle for classique.
Exemple : nombres = [x * 2 for x in [1, 2, 3, 4]]
# Résultat : [2, 4, 6, 8]

```
>>> m[3,5]
35
```

élément en position (3,5)

```
>>> m[3]
array([30, 31, 32, 33, 34, 35, 36, 37])
```

vecteur-ligne en position 3

```
>>> m[:,5]
array([ 5, 15, 25, 35, 45])
```

vecteur-colonne en position 5

```
>>> m[1:4,2:6]
array([[12, 13, 14, 15],
       [22, 23, 24, 25],
       [32, 33, 34, 35]])
```

lignes 1 à 3, colonnes 2 à 5

```
>>> m[1::2,1::2]
array([[11, 13, 15, 17],
       [31, 33, 35, 37]])
```

lignes et colonnes impaires

```
>>> m[::2,::2]
array([[ 0,  2,  4,  6],
       [20, 22, 24, 26],
       [40, 42, 44, 46]])
```

lignes et colonnes paires

```
>>> m[:3,:2]
array([[ 0,  1],
       [10, 11],
       [20, 21]])
```

trois premières lignes
deux premières colonnes

```
>>> m[2:,4:]
array([[24, 25, 26, 27],
       [34, 35, 36, 37],
       [44, 45, 46, 47]])
```

à partir de la ligne 2
à partir de la colonne 4

```
>>> m[::2,::3]
array([[ 0,  3,  6],
       [20, 23, 26],
       [40, 43, 46]])
```

une ligne sur deux
une colonne sur trois

```
>>> m[::-1]
array([[40, 41, 42, 43, 44, 45, 46, 47],
       [30, 31, 32, 33, 34, 35, 36, 37],
       [20, 21, 22, 23, 24, 25, 26, 27],
       [10, 11, 12, 13, 14, 15, 16, 17],
       [ 0,  1,  2,  3,  4,  5,  6,  7]])
```

on inverse l'ordre des lignes

```
>>> m[:,::-1]
array([[ 7,  6,  5,  4,  3,  2,  1,  0],
       [17, 16, 15, 14, 13, 12, 11, 10],
       [27, 26, 25, 24, 23, 22, 21, 20],
       [37, 36, 35, 34, 33, 32, 31, 30],
       [47, 46, 45, 44, 43, 42, 41, 40]])
```

on inverse l'ordre des colonnes


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Écriture de valeurs dans un vecteur

- Si $a$ est un vecteur numpy l'instruction `a[n] = x` écrit la valeur $x$ en position $n$.

```
>>> a = np.arange(10) # le vecteur des entiers de 0 à 9
>>> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> a[7] = 999 # écrit la valeur 999 en position 7
>>> a
array([ 0, 1, 2, 3, 4, 5, 6, 999, 8, 9])
```

- La valeur $x$ écrite en position $n$ du vecteur a doit a priori être compatible avec le « data type » de a.
- Si on écrit un entier x dans un tableau a de flottants, pas de problème (x est converti en le flottant correspondant).
- Mais si on écrit un flottant x dans un tableau a d'entiers, alors x est converti en un entier (par troncature, pas par
- arrondi) pour l'écriture : il n'y a donc pas de message d'erreur, ni même d'avertissement. Le « data type » et l'adresse du
- tableau a ne changent pas.

# Écriture de valeurs dans un vecteur

- La valeur $x$ écrite en position $n$ du vecteur $a$ doit a priori être compatible avec le « data type » de $a$.
  - Si on écrit un **entier** $x$ dans un tableau $a$ de **flottants**, pas de problème ($x$ est converti en le **flottant** correspondant).
  - Mais si on écrit un **flottant** $x$ dans un tableau $a$ **d'entiers**, alors $x$ est converti en un **entier** (par troncature, pas par arrondi) pour l'écriture : il n'y a donc pas de message d'erreur, ni même d'avertissement. Le « data type » et l'adresse du tableau a ne changent pas.
- L'idée fondamentale avec les tableaux numpy est que leur **data type** et leur **taille** (le nombre total d'éléments) sont fixés lors de leur création.
  - Par souci d'efficacité et d'économie, Python limite la création de copies du tableau initial, et donc privilégie les opérations réalisées « en place ».
  - Dans l'exemple ci-dessous, on voit que l'instruction `a[5]=12.99` (où $a$ est un tableau d'entiers) a été traduite en `a[5]=12` : l'opération a été effectuée « en place » (l'adresse du tableau en mémoire n'a pas changé).

```
>>> a = np.arange(10)
>>> a, id(a) # demande le contenu du tableau a, et son adresse
(array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]), 3957296)
>>> a[5] = 12.99 # on essaie d'écrire le flottant 12.99 en position 5
>>> a, id(a) # en fait, on a écrit 12, et l'adresse de a est inchangée
(array([ 0, 1, 2, 3, 4, 12, 6, 7, 8, 9]), 3957296)
```

# Écriture de valeurs dans un vecteur

- On peut écrire plusieurs valeurs simultanément dans un vecteur, en utilisant une syntaxe du genre `a[coupe ]=....` Il faut simplement veiller à ce que le membre de droite (la source) soit « array-like » (une liste, un tuple, un tableau) et que la coupe spécifiée *a* (donc la cible) soient de même taille. Ainsi, dans l'exemple ci-dessous, les instructions `a[4:7] = [111,222,333], a[4:7] = (111,222,333)` ou encore `a[4:7] = np.array([111,222,333])` ont le même effet :

```
>>> a = np.arange(10); a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> a[4:7] = [111,222,333]; a # place 111, 222, 333 en positions respectives 4, 5, 6
array([0, 1, 2, 3, 111, 222, 333, 7, 8, 9])
```

- L'exemple suivant est peut-être moins évident. On commence par former un vecteur nul de longueur 10 (avec le data type par défaut, donc float), puis on évalue l'instruction `a[::2] = np.arange(1,6)`. On affecte donc respectivement les cinq valeurs du tableau `array([1, 2, 3, 4, 5])` aux positions paires du tableau *a*.

```
>>> a = np.zeros(10); a, id(a) # un vecteur de 10 flottants égaux à 0, et son adresse
(array([ 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]), 19048976)
>>> a[::2] = np.arange(1,6); a,i d(a) # place 1,2,3,4,5 aux positions 0,2,4,6,8. Adresse inchangée
(array([ 1., 0., 2., 0., 3., 0., 4., 0., 5., 0.]), 19048976)
```

# Écriture de valeurs dans un vecteur

- Autre exemple un peu tordu, on copie le vecteur *a* à l'envers sur lui-même :

```
>>> a = np.arange(10); a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> a[::-1] = a; a
array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

# Écriture de valeurs dans une matrice

- On procède comme pour les vecteurs, indice par indice.

```
>>> m
array([[ 0, 1, 2, 3],
       [10, 11, 12, 13],
       [20, 21, 22, 23]])
```

- Commençons par l'écriture d'un coefficient :

```
m[1,2] = 999; m
array([[ 0, 1, 2, 3],
       [ 10, 11, 999, 13],
       [ 20, 21, 22, 23]])
```

- Continuons par l'écriture d'une ligne entière :

```
>>> m[2] = [6, 7, 8, 9]; m
array([[ 0, 1, 2, 3],
       [10, 11, 12, 13],
       [ 6, 7, 8, 9]])
```

- Terminons par l'écriture d'une colonne entière.

```
>>> m[:,1] = [444, 555, 666]; m
array([[ 0, 444, 2, 3],
       [ 10, 555, 12, 13],
       [ 20, 666, 22, 23]])
```

- Il y a énormément de possibilités, mais en dehors des trois précédentes, il faut vraiment en avoir l'utilité.
- Juste pour réviser un peu l'utilisation de coupes à l'envers `::-1`, citons les exemples suivants :

```
>>> m[::-1] = m; m
array([[20, 21, 22, 23],
       [10, 11, 12, 13],
       [ 0, 1, 2, 3]])
```

on a inversé l'ordre des lignes

```
>>> m[:,::-1] = m; m
array([[23, 22, 21, 20],
       [13, 12, 11, 10],
       [ 3, 2, 1, 0]])
```

inversé l'ordre des colonnes

# Copies de tableaux, « vues » sur des tableaux

- Avec les tableaux numpy, on applique donc un principe d'économie qui consiste à effectuer aussi **peu** que possible de **recopies** de tableaux en mémoire, sauf demande explicite. Tout dépend de la fonction utilisée sur le tableau.
- L'exemple suivant est très important (**parce que la situation évoquée ici revient souvent**). Le vecteur *a* contenant un tableau numpy, la variable *a* pointe en fait sur la position de ce tableau en mémoire. L'instruction `b = a` ne crée pas une nouvelle copie du tableau *a*, mais elle demande à la variable *b* de pointer sur le même tableau physique que *a*.
- On exprime cette situation en disant que *a* et *b* constituent une même vue (« view » in english) d'un tableau numpy.
- La conséquence immédiate est que toute modification apportée à cette *vue* (que ce soit par l'intermédiaire de la variable *a* ou de la variable *b*) affecte ce que « voient » et *a* et *b*. Ici, par exemple, après avoir posé *b* = *a*, l'instruction b[2,1]=9 modifie le tableau vu par *b*, c'est-à-dire le tableau vu par *a*. C'est confirmé quand on évalue les expressions id(a) et id(b) qui donnent un résultat identique (l'adresse de ce que voient *a* et *b*).

```
>>> a
array([[ 0, 1, 2, 3],
       [10,11,12,13],
       [20,21,22,23]])
>>> b = a
```

```
>>> b[2,1] = 9; b
array([[ 0, 1, 2, 3],
       [10,11,12,13],
       [20, 9,22,23]])
```

on modifie `b[2,1]` dans le tableau *b*

```
>>> a
array([[ 0, 1, 2, 3],
       [10,11,12,13],
       [20, 9,22,23]])
```

la modification s'est répercutée sur *a*

```
>>> id(a)
19127280
>>> id(b)
19127280
```

c'est normal car *a* et *b* partagent une même vue


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Copies de tableaux, « vues » sur des tableaux

- Il est bien sûr possible de « déconnecter » complètement les tableaux vus respectivement par deux identificateurs.
- L'expression `a.copy()` renvoie une copie « neuve » du tableau *a*, indépendante de l'original.

```
>>> a
array([[ 0, 1, 2, 3],
       [10,11,12,13],
       [20,21,22,23]])
>>> b = a.copy()
```

```
>>> b[2,1] = 9; b
array([[ 0, 1, 2, 3],
       [10,11,12,13],
       [20, 9,22,23]])
```

on modifie `b[2,1]` dans le tableau *b*

```
>>> a
array([[ 0, 1, 2, 3],
       [10,11,12,13],
       [20,21,22,23]])
```

la modification ne s'est par répercutée sur *a*

```
>>> id(a)
19134464
>>> id(b)
19096112
```

normal car *a* et *b* ne partagent pas la même vue

# Copies de tableaux, « vues » sur des tableaux

- L'exemple suivant est beaucoup plus subtil et intéressant et instructif !

```
>>> a
array([[ 0,  1,  2,  3,  4,  5,  6,  7],
       [10, 11, 12, 13, 14, 15, 16, 17],
       [20, 21, 22, 23, 24, 25, 26, 27],
       [30, 31, 32, 33, 34, 35, 36, 37],
       [40, 41, 42, 43, 44, 45, 46, 47]])
```

1

la matrice $a$ initiale

```
>>> b = a[1:4,2:6]
>>> b
array([[12, 13, 14, 15],
       [22, 23, 24, 25],
       [32, 33, 34, 35]])
```

2

on en extrait une sous-matrice $b$

```
>>> b.fill(0)
>>> b
array([[0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0]])
```

3

on annule les coefficients de $b$

- Ce qui est plus inattendu, c'est que tous les coefficients de $a$ et qui correspondant à l'emprunt que nous avons fait pour créer $b$ sont annulés eux aussi ! ça se confirme après l'instruction `b.fill(1)` qui remplit de coefficients égaux à 1 le tableau vu par $b$, c'est-à-dire la sous-matrice de a correspondand à la définition de b.

```
>>> a
array([[ 0,  1,  2,  3,  4,  5,  6,  7],
       [10, 11,  0,  0,  0,  0, 16, 17],
       [20, 21,  0,  0,  0,  0, 26, 27],
       [30, 31,  0,  0,  0,  0, 36, 37],
       [40, 41, 42, 43, 44, 45, 46, 47]])
```

annuler $b$, c'est annuler une vue
sur la partie du tableau $a$ dont $b$ est issue

```
>>> b.fill(1); a
array([[ 0,  1,  2,  3,  4,  5,  6,  7],
       [10, 11,  1,  1,  1,  1, 16, 17],
       [20, 21,  1,  1,  1,  1, 26, 27],
       [30, 31,  1,  1,  1,  1, 36, 37],
       [40, 41, 42, 43, 44, 45, 46, 47]])
```

c'est la même chose en remplissant la matrice $b$
de coefficients égaux à 1

# Copies de tableaux, « vues » sur des tableaux

- **Important** : on retiendra donc que lire une coupe d'un tableau numpy *a*, ça n'est pas créer un nouveau tableau, c'est renvoyer une « vue » sur un sous-tableau de *a*.

# Exercice

**Ex. 1** — Répondre à chacune des questions avec **une seule ligne** de code.

1. Créer un tableau A qui contient tous les multiples de 3 entre 0 et 100.

2. Créer un tableau B qui contient les sinus des carrés des éléments de A multipliés par $\pi/12$.

3. Déterminer le maximum du tableau B et le garder dans une variable.

4. Compter le nombre de fois que ce maximum est présent dans B. Commentaire.

# Exercice

**Ex. 1** — Répondre à chacune des questions avec une seule ligne de code.

1. Créer un tableau A qui contient tous les multiples de 3 entre 0 et 100.

2. Créer un tableau B qui contient les sinus des carrés des éléments de A multipliés par π/12.

3. Déterminer le maximum du tableau B et le garder dans une variable.

4. Compter le nombre de fois que ce maximum est présent dans B. Commentaire.

**Corrigé de l'exercice 1:**

```
A = np. arange (0 ,100 ,3)
B = np.sin (A**2 * math .pi /12)
m = np.max(B)
np.sum(B == m)
```

# Fancy indexing

- **Fancy indexing** (ou *indexation fantaisie*) permet d'accéder à plusieurs éléments d'un tableau NumPy **en une seule opération**,
  en utilisant :
  - une **liste** ou un **tableau d'indices**,
  - ou encore un **masque booléen**.
- On peut accéder au contenu d'un tableau numpy tant en lecture qu'en écriture, en utilisant le « fancy indexing » (selon la terminologie officielle…). Ici les positions dans un vecteur ou dans une matrice ne procèdent plus nécessairement par des « coupes », mais peuvent être données dans un ordre tout à fait quelconque.
- Si *a* est un tableau, et si *I* est un tableau d'indices (éventuellement une liste ou un tuple), alors `a[I]` renvoie le tableau (de même format que le tableau *I*) formé des éléments `a[i]`, où `i` est dans *I*.

```
>>> a = np.array([10, 20, 30, 40, 50])
>>> I = [0, 2, 4]
>>> result = a[I] # fancy indexing
>>> result
array([10, 30, 50])
```

# Fancy indexing

```
>>> a = np.arange(0,100,10); a
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90])
>>> b = a[[6,2,1,3]]; b
array([60, 20, 10, 30])
```

on demande le tableau $[a_6, a_2, a_1, a_3]$

```
>>> indices = np.array([[5,2,1],[3,8,7]])
>>> b = a[indices]; b
array([[50, 20, 10],
[30, 80, 70]])
```

on demande le tableau $[[a_5, a_2, a_1]$
$[a_3, a_8, a_7]]$

```
>>> a = np.arange(24).reshape(4,6); a
array([[ 0, 1, 2, 3, 4, 5],
       [ 6, 7, 8, 9, 10, 11],
       [12, 13, 14, 15, 16, 17],
       [18, 19, 20, 21, 22, 23]])
```

```
>>> b = a[[3,1,0,1]]; b
array([[18, 19, 20, 21, 22, 23],
[ 6, 7, 8, 9, 10, 11],
[ 0, 1, 2, 3, 4, 5],
[ 6, 7, 8, 9, 10, 11]])
```

on demande le tableau des lignes $a_3, a_1, a_0, a_1$

Subtilité : les tableaux *b* formés ci-dessus (et qui ne sont pas obtenus par « slicing ») ne sont pas de simples « vues » sur une partie de *a*. Ils sont donc indépendants de *a* (modifier *b* n'affecte pas *a* et réciproquement).

# Fancy indexing

- Le « fancy indexing », ça fonctionne aussi pour des opérations d'écriture :

```
>>> a = np.arange(0,100,10); a
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90])
>>> a[[7, 2, 5, 3]] = (7777, 2222, 5555, 3333); a # on modifie a[7], a[2], a[5] et a[3]
array([ 0, 10, 2222, 3333, 40, 5555, 60, 7777, 80, 90])
```

# Dimensions d’un tableau



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Redimensionnement par « reshape » ou « resize »

- La principale méthode pour modifier la taille d'un tableau est `reshape` .
- Le redimensionnement d'un tableau est quand même soumis à la contrainte que le nombre total d'éléments doit rester constant.
- On pourra ainsi passer (dans les deux sens) d'un vecteur de taille $n$ à une matrice de taille $(p, q)$ ou à une matrice de taille $(r, s)$ à condition que $n = pq = rs$.
- Si $a$ est un tableau numpy, l'expression `a.reshape(n,p)` renvoie une copie **redimensionnée** (le contenu de la variable initiale n'est donc pas affecté, comme on le constate ci-dessous).

```
>>> a = np.array(range(6)); a
array([0, 1, 2, 3, 4, 5])
>>> a.reshape(1,6)
array([[0, 1, 2, 3, 4, 5]])
>>> a
array([0, 1, 2, 3, 4, 5])
```

```
>>> a.reshape(2,3)
array([[0, 1, 2],
       [3, 4, 5]])
>>> a.reshape(3,2)
array([[0, 1],
       [2, 3],
       [4, 5]])
```

```
>>> a.reshape(6,1)
array([[0],
       [1],
       [2],
       [3],
       [4],
       [5]])
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Redimensionnement par « reshape » ou « resize »

- Il y a une autre possibilité qui consiste à réinitialiser *l'attribut* shape d'un tableau numpy.
- La différence avec ce qui précède est que le redimensionnement est fait « sur place », et donc que le contenu de la variable s'en trouve affecté.

```
>>> a = np.array(range(6))
>>> a
array([0, 1, 2, 3, 4, 5])
>>> a.shape = (2,3); a # a devient matrice 2 × 3
array([[0, 1, 2],
       [3, 4, 5]])
```

```
>>> a.shape = (3,2); a # a devient matrice 3 × 2
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> a.shape = 6; a # redevient un vecteur
array([0, 1, 2, 3, 4, 5])
```

# Redimensionnement par « reshape » ou « resize »

- Autre possibilité, la fonction `resize` , qui permet de **redimensionner** un bloc, en transformant (sur place) un tableau *T* (à *n* éléments ) en un tableau *T'* (à *n'* éléments). Si *n'* > *n*, le tableau *T'* est rempli avec répétitions des éléments de *T*. Sinon *T'* est rempli par les *n* premiers éléments de *T*.

```
>>> a = np.array(range(6)); a
array([0, 1, 2, 3, 4, 5])
>>> a.resize(3,2); a
array([[0, 1],
[2, 3],
[4, 5]])
>>> a.resize(2,3); a
array([[0, 1, 2],
       [3, 4, 5]])
```

avec `a.resize` la modification s'effectue « en place »

```
>>> np.resize(a,(4,3))
array([[0, 1, 2],
       [3, 4, 5],
       [0, 1, 2],
       [3, 4, 5]])
>>> a
array([[0, 1, 2],
       [3, 4, 5]])
```

avec `np.resize(a,...)`, un nouveau tableau est créé

```
>>> np.resize(a,(1,7))
array([[0, 1, 2, 3, 4, 5, 0]])
>>> np.resize(a,(2,5))
array([[0, 1, 2, 3, 4],
       [5, 0, 1, 2, 3]])
>>> a
array([[0, 1, 2],
       [3, 4, 5]])
```

autres exemples avec `np.resize(a,...)`

# Redimensionnement par « reshape » ou « resize »

- La fonction `resize` permet également de créer des tableaux constants contenant une autre valeur que 0 et 1 (sinon on utiliserait les fonctions `zeros` ou `ones`) :

```
>>> np.resize(1.23, 4)
array([ 1.23, 1.23, 1.23, 1.23])
>>> np.resize(9, 7)
array([9, 9, 9, 9, 9, 9, 9])
```

```
>>> np.resize(5, (3,6))
array([[5, 5, 5, 5, 5, 5],
[5, 5, 5, 5, 5, 5],
[5, 5, 5, 5, 5, 5]])
```

# Aplatissement d'un tableau

- La méthode `flatten` d'un tableau numpy renvoie une copie « aplatie » de ce tableau.
- Cette fonction est l'occasion de découvrir un argument facultatif souvent employé quand on est amené à parcourir un tableau. C'est l'argument `order=...` dont la valeur par défaut est `order= 'C'` et qui peut être modifié en `order= 'F'`.
- Avec `order='C'` (où `C` désigne le langage de programmation) le parcours s'effectue d'abord de gauche à droite sur une ligne avant de passer à la ligne suivante (donc l'indice de colonne varie prioritairement). Avec `order='F'` (où `F` désigne le langage de programmation Fortran) le parcours s'effectue d'abord de haut en bas sur une colonne avant de passer à la colonne suivante (donc l'indice de ligne varie prioritairement).

```
>>> a = np.array([[1,5,3],[2,7,4]])
>>> a
array([[1, 5, 3],
       [2, 7, 4]])
```

```
>>> a.flatten() # parcours par défaut (à droite d'abord)
array([1, 5, 3, 2, 7, 4])
>>> a.flatten('F')# parcours Fortran (en bas d'abord)
array([1, 2, 5, 7, 3, 4])
```

# Transposition d'une matrice

- La transposée d'une matrice consiste à échanger ses lignes et ses colonnes.
  - Autrement dit :

$$A_{ij} \rightarrow A_{ji}$$

  - Exemple :

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

- La fonction (ou la méthode) `transpose` , ou plus simplement la méthode `T` , renvoient la transposée d'une matrice :

```
>>> a
array([[ 1, 2, 3, 4],
       [ 5, 6, 7, 8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16]])
```

```
>>> a.transpose()
array([[ 1, 5, 9, 13],
       [ 2, 6, 10, 14],
       [ 3, 7, 11, 15],
       [ 4, 8, 12, 16]])
```

```
>>> a.T
array([[ 1, 5, 9, 13],
       [ 2, 6, 10, 14],
       [ 3, 7, 11, 15],
       [ 4, 8, 12, 16]])
```

# Suppressions/insertions de lignes, de colonnes

- En **NumPy**, les tableaux (ndarray) ont une taille **fixe**, donc on ne peut pas "ajouter" ou "supprimer" **directement** un élément comme dans une liste Python.
- Mais NumPy fournit des fonctions qui créent un nouveau tableau modifié, ce qui revient au même du point de vue pratique.

```
>>> a = np.array([1, 2, 3])
>>> a
array([1, 2, 3])
>>> a = np.append(a, 4)
>>> a
array([1, 2, 3, 4])
>>> a = np.append(a, [5, 6])
>>> a
array([1, 2, 3, 4, 5, 6])
```

```
>>> a = np.array([10, 20, 30, 40, 50])
>>> a = np.delete(a, 2)
>>> a
array([10, 20, 40, 50])
>>> a = np.delete(a, [0, 2])
>>> a
array([20, 50])
```

# Suppressions/insertions de lignes, de colonnes

- `delete(a,k,axis=0)` et `delete(a,k,axis=1)` suppriment respectivement la *k*-ième ligne et la *k*-ième colonne de *a*.
- Le troisième argument indique donc le numéro d'indice (*l'axe*) selon lequel on pratique cette suppression
- L'argument `axis=...` est très souvent présent (de façon facultative) dans les fonctions du module numpy : disons pour simplifier, que pour des matrices (deux indices), `axis=0` signifie « selon l'indice de **ligne** » et axis=1 signifie « selon l'indice de **colonne** ». Souvent, d'ailleurs, le mot axis peut être omis.

```
>>> a
array([[ 0, 1, 2, 3, 4],
       [10, 11, 12, 13, 14],
       [20, 21, 22, 23, 24],
       [30, 31, 32, 33, 34]])
```

une *a* matrice de type 4 × 5

```
>>> b = np.delete(a,2,0)
>>> b
array([[ 0, 1, 2, 3, 4],
       [10, 11, 12, 13, 14],
       [30, 31, 32, 33, 34]])
```

la matrice *b* est obtenue en supprimant la ligne d'indice 2 de *a*

```
>>> c = np.delete(b,3,1)
>>> c
array([[ 0, 1, 2, 4],
       [10, 11, 12, 14],
       [30, 31, 32, 34]])
```

la matrice *c* est obtenue en supprimant la colonne d'indice 3 de *b*

# Suppressions/insertions de lignes, de colonnes

- On peut également supprimer plusieurs colonnes (ou lignes) à la fois. Voici quelques exemples :

```
>>> a
array([[ 0, 1, 2, 3, 4],
       [10, 11, 12, 13, 14],
       [20, 21, 22, 23, 24],
       [30, 31, 32, 33, 34]])
```

```
>>> np.delete(a,np.s_[0::2],1)
array([[ 1,  3],
       [11, 13],
       [21, 23],
       [31, 33]])
```

supprime les colonnes d'indice pair

```
>>> np.delete(a,np.s_[1::2],1)
array([[ 0,  2,  4],
       [10, 12, 14],
       [20, 22, 24],
       [30, 32, 34]])
```

supprime les colonnes d'indice impair

```
>>> np.delete(a,[0,2,3],1)
array([[ 1,  4],
       [11, 14],
       [21, 24],
       [31, 34]])
```

supprime les colonnes d'indice 0, 2, 3

# Suppressions/insertions de lignes, de colonnes

- Les expressions `insert(a,k,v,axis=0)` et `insert(a,k,v,axis=1)` permettent d'insérer la valeur $v$ respectivement avant la $k$-ième ligne ou avant la $k$-ième colonne de $a$. Voici deux exemples :

```
>>> a
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23]])
```

une matrice $a$ de type $3 \times 4$

```
>>> np.insert(a,2,-1,axis=1)
array([[ 0,  1, -1,  2,  3],
       [10, 11, -1, 12, 13],
       [20, 21, -1, 22, 23]])
```

insère des $-1$ juste avant la colonne d'indice 2

```
>>> np.insert(a,1,0,axis=0)
array([[ 0,  1,  2,  3],
       [ 0,  0,  0,  0],
       [10, 11, 12, 13],
       [20, 21, 22, 23]])
```

insère des 0 juste avant la ligne d'indice 1

# Suppressions/insertions de lignes, de colonnes

- Les expressions `append(a,v,axis=0)` et `append(a,v,axis=1)` permettent d'ajouter une matrice-colonne $C$ (respectivement une matrice $V$) après la dernière ligne (respectivement après la dernière colonne) de $a$. Il faut veiller à ce que la matrice ajoutée ait un format compatible avec celui de $a$. Voici deux exemples :

```
>>> a
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23]])
```

une matrice $a$ de type $3 \times 4$

```
>>> np.append(a,[[7],[8],[9]],1)
array([[ 0,  1,  2,  3,  7],
       [10, 11, 12, 13,  8],
       [20, 21, 22, 23,  9]])
```

ajoute une colonne [[7,8,9]] après la dernière colonne

```
>>> np.append(a,[[3,5,7,9]],0)
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [ 3,  5,  7,  9]])
```

ajoute une ligne [3,5,7,9] après la dernière ligne

# Opérations par blocs

- `concatenate` permet d'accoler deux ou plusieurs tableaux (horizontalement avec `axis=1`, verticalement avec `axis=0`).
- En cas de concaténation horizontale par exemple, toutes les colonnes doivent avoir la même hauteur.
- Les différents tableaux à concaténer doivent être donnés sous la forme d'un tuple :

```
>>> a = np.array(range(8)).reshape(2,4)
>>> a
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])

>>> b = np.zeros(a.shape,int); b
array([[0, 0, 0, 0],
       [0, 0, 0, 0]])

>>> c = np.ones(a.shape,int); c
array([[1, 1, 1, 1],
       [1, 1, 1, 1]])
```

```
>>> np.concatenate((a,b),axis=1)
array([[0, 1, 2, 3, 0, 0, 0, 0],
       [4, 5, 6, 7, 0, 0, 0, 0]])
>>> np.concatenate((a,b,c),axis=1)
array([[0, 1, 2, 3, 0, 0, 0, 0, 1, 1, 1, 1],
       [4, 5, 6, 7, 0, 0, 0, 0, 1, 1, 1, 1]])
>>> np.concatenate((a,c),axis=0)
array([[0, 1, 2, 3],
       [4, 5, 6, 7],
       [1, 1, 1, 1],
       [1, 1, 1, 1]])
```

# Opérations par blocs

- On peut aussi utiliser les fonctions `hstack` (concaténation horizontale) et `vstack` (concaténation verticale).
- Voici quelques exemples en reprenant la signification précédente des tableaux *a, b, c* :

```
>>> np.vstack((a,c))
array([[0, 1, 2, 3],
       [4, 5, 6, 7],
       [1, 1, 1, 1],
       [1, 1, 1, 1]])
```

```
>>> np.hstack((a,c))
array([[0, 1, 2, 3, 1, 1, 1, 1],
       [4, 5, 6, 7, 1, 1, 1, 1]])
>>> np.hstack((a,b,c))
array([[0, 1, 2, 3, 0, 0, 0, 0, 1, 1, 1, 1],
       [4, 5, 6, 7, 0, 0, 0, 0, 1, 1, 1, 1]])
```

# Exercice

1. Crée un tableau *a* contenant les entiers de 0 à 11.
2. Redimensionne-le en une matrice *b* = 3×4 avec reshape.
3. Redimensionne-le en 2×6 sur place avec resize.
4. Modifiez une valeur de a (par exemple `a[0, 0] = 99`)
5. Vérifiez les valeurs de a et b, que constatez-vous ?
6. À partir de la matrice *b (3×4)* précédente, crée un tableau 1D contenant les mêmes éléments.
7. Transpose la matrice *b (3×4)* et affiche le résultat.
8. Supprime la 1re ligne et la dernière colonne de *b*.
9. Insère une nouvelle colonne `[100, 200, 300]` à la fin.

# Exercice - Solution

1. `a = np.arange(12)`
2. `b = a.reshape(3, 4)`
3. `a.resize(2, 6)`
4. `a[0, 0] = 99`
5. Comparaison a b
   - reshape() crée une nouvelle vue du tableau original : b dépend encore de a, mais ne le modifie pas directement.
   - resize() modifie directement le tableau a.
6. `flat = b.flatten()` : flatten() crée une **copie**
7. `b.T`
8. Suppression
   - `c = np.delete(b, 0, axis=0)  # supprime ligne 0`
   - `c = np.delete(c, -1, axis=1)  # supprime dernière colonne`
9. Insertion
   - `new_col = np.array([100, 200, 300])`
   - `b2 = np.insert(b, b.shape[1], new_col, axis=1)`


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Tableaux spécifiques



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Tableaux constants

- La fonction `zeros` permet de former des tableaux dont tous les coefficients sont nuls.
  - Le premier argument, obligatoire, précise le format (*shape*) du tableau : un entier $n$ pour un vecteur de longueur $n$, un couple $(n, p)$ pour une matrice de type $n \times p$, ou encore un tuple $(n, p, q, r, \ldots)$ pour un tableau à plus de deux indices.

Attention : pour créer des tableaux 2d et plus, les dimensions doivent être données sous la forme d'un seul argument tuple. La syntaxe zeros(3,5) est incorrecte, il faut écrire `zeros((3,5))`.

  - Un argument facultatif permet de fixer le type de données (c'est float par défaut).

```
>>> np.zeros(5)
array([ 0.,  0.,  0.,  0.,  0.])
>>> np.zeros((1,5))
array([[ 0.,  0.,  0.,  0.,  0.]])
>>> np.zeros(5,int)
array([0, 0, 0, 0, 0])
```

```
>>> np.zeros((5,3))
array([[ 0.,  0.,  0.],
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.]])
```

```
>>> np.zeros((5,1),complex)
array([[ 0.+0.j],
       [ 0.+0.j],
       [ 0.+0.j],
       [ 0.+0.j],
       [ 0.+0.j]])
```

```
>>> np.zeros((2,2,2)) # tableau 3d de taille 2 x 2 x 2 contenant des 0
# array([[[ 0., 0.],[ 0., 0.]],
#        [[ 0., 0.], [ 0., 0.]]])
```

# Tableaux constants

- Avec une syntaxe analogue, la fonction `ones` permet de former des tableaux dont tous les coefficients valent `1`.

```
>>> np.ones((3,6))
array([[ 1.,  1.,  1.,  1.,  1.,  1.],
       [ 1.,  1.,  1.,  1.,  1.,  1.],
       [ 1.,  1.,  1.,  1.,  1.,  1.]])
```

```
>>> np.ones((3,6),int)
array([[1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1]])
```

- Rappelons que chez les booléens, `false = 0 = « nul » = « vide »`, et `True = 1 = « non nul » = « non vide »` :

```
>>> np.zeros((3,3),bool)
array([[False, False, False],
       [False, False, False],
       [False, False, False]], dtype=bool)
```

```
>>> np.ones((3,3),bool)
array([[ True,  True,  True],
       [ True,  True,  True],
       [ True,  True,  True]], dtype=bool)
```

# Tableaux constants

- La méthode `fill` d'un tableau numpy remplit ce tableau par une valeur constante.
- Comme le montrent les exemples suivants, le remplissage se fait « en place ». L'adresse du tableau en mémoire ne change pas. En particulier, il n'est pas possible de modifier le « data type » (voir la colonne de droite, ou la valeur flottante 3.14 a été convertie en la valeur entière 3 pour le remplissage).

```
>>> a = np.array([[1,5,3],[2,7,4]])
>>> a, id(a)
(array([[1, 5, 3],
        [2, 7, 4]]), 3899264)
```

```
>>> a.fill(1);
>>> a, id(a)
(array([[1, 1, 1],
        [1, 1, 1]]), 3899264)
```

```
>>> a.fill(3.14);
>>> a, id(a)
(array([[3, 3, 3],
        [3, 3, 3]]), 3899264)
```

`full(shape, valeur)` → Crée un nouveau tableau rempli avec une valeur donnée.

`fill(valeur)` → Modifie un tableau existant sur place (in-place).

# Identité

- Une matrice identité. C'est une matrice carrée avec :
  - des 1 sur la diagonale principale,
  - des 0 partout ailleur
- La fonction `identity` fabrique la matrice identité d'ordre $n$ (donné comme premier argument).

```
>>> np.identity(4)
array([[ 1., 0., 0., 0.],
       [ 0., 1., 0., 0.],
       [ 0., 0., 1., 0.],
       [ 0., 0., 0., 1.]])
```

```
>>> np.identity(4,int)
array([[1, 0, 0, 0],
       [0, 1, 0, 0],
       [0, 0, 1, 0],
       [0, 0, 0, 1]])
```

```
>>> np.identity(4,complex)
array([[ 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [ 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j],
       [ 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j],
       [ 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j]])
```

# Identité

- La fonction `eye` permet de fabriquer la matrice identité ou plus généralement une matrice dont tous les coefficients sont nuls sauf ceux d'une certaine « parallèle » à la diagonale et qui valent 1.

  `np.eye(n, p=None, k=0)`
  - n : Nombre de lignes
  - p : Nombre de colonnes (optionnel, = n si non précisé)
  - k : Décalage de la diagonale (0 = principale, +1 = au-dessus, -1 = en dessous)
- Le premier argument (obligatoire) donne le nombre $n$ de lignes. Le second argument (facultatif) donne le nombre $p$ de colonnes (par défaut $p = n$ donc la matrice est carrée).
- Un argument facultatif nommé `k = ..` permet de spécifier un décalage de la diagonale de 1 au dessus (si $k > 0$) ou en-dessous (si $k < 0$) de la diagonale principale.
- Enfin, on peut ajouter un argument pour fixer le type des données (float par défaut).

```
>>> np.eye(5,dtype=int)
array([[1, 0, 0, 0, 0],
       [0, 1, 0, 0, 0],
       [0, 0, 1, 0, 0],
       [0, 0, 0, 1, 0],
       [0, 0, 0, 0, 1]])
```

```
>>> np.eye(5,6,dtype=int)
array([[1, 0, 0, 0, 0, 0],
       [0, 1, 0, 0, 0, 0],
       [0, 0, 1, 0, 0, 0],
       [0, 0, 0, 1, 0, 0],
       [0, 0, 0, 0, 1, 0]])
```

```
>>> np.eye(1,5)
array([[ 1.,  0.,  0.,  0.,  0.]])
>>> np.eye(1,5,1)
array([[ 0.,  1.,  0.,  0.,  0.]])
>>> np.eye(1,5,2)
array([[ 0.,  0.,  1.,  0.,  0.]])
```

```
>>> np.eye(4,k=-2)
array([[ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.],
       [ 1.,  0.,  0.,  0.],
       [ 0.,  1.,  0.,  0.]])
```

```
>>> np.eye(4,k=1)
array([[ 0.,  1.,  0.,  0.],
       [ 0.,  0.,  1.,  0.],
       [ 0.,  0.,  0.,  1.],
       [ 0.,  0.,  0.,  0.]])
```

```
>>> np.eye(4,k=3)
array([[ 0.,  0.,  0.,  1.],
       [ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.]])
```

# Tableaux de valeurs échelonnées

- La fonction `arange` crée des vecteurs (1d) de valeurs régulièrement espacées.
- La syntaxe est `arange(start, stop, step,dtype=...)`, et génère les valeurs de l'intervalle `[start, stop[` avec un pas de `step`.

```
>>> np.arange(10)
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.arange(100,200,20)
array([100, 120, 140, 160, 180])
>>> np.arange(100,201,20)
array([100, 120, 140, 160, 180, 200])
```

```
>>> np.arange(10,0,-1)
array([10, 9, 8, 7, 6, 5, 4, 3, 2, 1])
>>> np.arange(8,dtype=float)
array([ 0., 1., 2., 3., 4., 5., 6., 7.])
>>> np.arange(8.)
array([ 0., 1., 2., 3., 4., 5., 6., 7.])
```

- Pour les valeurs dans le type `float` ou `complex`, et pour éviter une erreur d'arrondi qui pourrait affecter la dernière valeur de l'intervalle, il peut être préférable d'utiliser la fonction `linspace` .

# Tableaux de valeurs échelonnées

- La syntaxe est `linspace(a,b,n)` et le résultat est un vecteur de $n$ valeurs régulièrement échelonnées du segment `[a, b]` (donc ici les extrémités sont incluses).
  - Par défaut, $n = 50$.
  - Si $b<a$, les valeurs sont obtenues dans l'ordre décroissant.

```
>>> np.linspace(0,100,4)
array([ 0., 33.33333333, 66.66666667, 100.])
>>> np.linspace(0,100,7)     # 7 valeurs de [0,100] dans l'ordre croissant
array([ 0., 16.66666667, 33.33333333, 50., 66.66666667, 83.33333333, 100.])
>>> np.linspace(100,0,7)     # 7 valeurs de [0,100] dans l'ordre décroissant
array([ 100., 83.33333333, 66.66666667, 50., 33.33333333, 16.66666667, 0.])
```

# Tableaux pseudo-aléatoires

Le sous-module numpy.random est une partie de du paquet numpy qui permet de générer des nombres pseudo-aléatoires selon différentes lois de probabilité.

- `random.rand` : Générer des **nombres** pseudo-aléatoires **réels** suivant une **loi uniforme** sur l'intervalle U (0, 1). Cela signifie que chaque valeur entre 0 et 1 a la **même probabilité** d'être choisie.

  ```
  numpy.random.rand(d1, d2, ...)
  ```

  où d1, d2 sont les dimensions du tableau souhaité.

- `random.randn` : Générer des **nombres réels** suivant une **loi normale centrée réduit**, c'est-à-dire une distribution gaussienne avec :**moyenne = 0** ; **écart-type = 1**

- `random.randint` : Générer des **nombres entiers** pseudo-aléatoires selon une **loi uniforme discrète** sur un intervalle donné. chaque valeur a la même probabilité d'être choisie.

  La syntaxe est `random.randint(a,b)` pour une seule valeur, `random.randint(a,b,n)` pour un vecteur de longueur $n$, et `random.randint(a,b,(n,p))` pour une matrice de format $n \times p$.

  Si on omet la valeur $a$, les entiers pseudo-aléatoires sont choisis dans `[0, b [`.

En informatique, les opérations sont déterministes : l'aléatoire n'existe pas, on parle de **pseudo-aléatoire**

# Tableaux pseudo-aléatoires

```
>>> np.random.rand(6)
array([ 0.14243652, 0.77345108, 0.76627661, 0.41079884, 0.79899263, 0.95726012])
>>> np.random.rand(2,6)
array([[ 0.5488135 , 0.71518937, 0.60276338, 0.54488318, 0.4236548 , 0.64589411],
[ 0.43758721, 0.891773 , 0.96366276, 0.38344152, 0.79172504, 0.52889492]])
```

```
>>> np.random.randn(6)
array([-0.75931933, 0.49230711, 0.37242323, 1.17957196, -2.26637722, 1.05330359])
```

```
>>> np.random.randint(100) # un seul entier pseudo-aléatoire dans [0,100[
72
>>> np.random.randint(1,7,10)
array([1, 3, 1, 5, 2, 6, 2, 4, 3, 6]) # dix lancers d'un dé à six faces
>>> np.random.randint(2,size=(2,8)) # un tableau de 2 × 8 entiers choisis dans {0, 1}
array([[1, 1, 0, 0, 0, 0, 0, 0],
       [1, 1, 1, 0, 1, 0, 1, 1]])
```

# Tableaux pseudo-aléatoires

**Graine ( : *seed*)**

- Pour la reproductibilité des résultats on peut souhaiter “geler” l’aléa (*e.g.*, pour des tests ou du débogage)
- **Graine** ( : *seed*) : nombre utilisé pour initialiser un générateur de nombres pseudo-aléatoires
- **Fonctionnement** : fixer une graine permet de spécifier de manière déterministe l’appel des opérations pseudo-aléatoires

```
>>> np.random.seed(2018) # fixe la graine
>>> np.random.rand(3, ) # tirage aléatoire
array([0.88234931, 0.10432774, 0.90700933])
```

**Résultat :** cette commande produit alors toujours la même sortie

# Tableaux pseudo-aléatoires

- La fonction `random.shuffle` rebat aléatoirement (et « en place ») les éléments d'un vecteur.

```
>>> a = np.array([1,7,3,8,5,7,1,2,7])            # un vecteur de 9 valeurs
>>> np.random.shuffle(a); a                      # rebat les éléments de a, sur place
array([7, 1, 8, 7, 1, 5, 2, 7, 3])
>>> a = np.arange(15); a                         # le vecteur des entiers de 0 à 14
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
>>> np.random.shuffle(a); a                      # rebat les éléments de a, sur place
array([ 0,  8,  9, 14,  3,  4,  2, 12, 10,  1,  6, 11, 13,  7,  5])
```

# Exercice

1. Crée un tableau A de taille $2\times3$ contenant des valeurs aléatoires uniformes comprises entre 0 et
2. Calcule la moyenne et la valeur maximale du tableau (A.mean() et A.max()).
3. Vérifie que toutes les valeurs sont bien comprises entre 0 et 1
4. Crée un tableau B de taille $4\times4$ à partir d'une loi normale centrée réduite (moyenne = 0, écart-type = 1).
5. Affiche la moyenne `(B.mean())` et l'écart-type `(B.std())` expérimentaux du tableau.
6. Compare ces valeurs à la moyenne et à l'écart-type théoriques (0 et 1).

# Exercice - Solution

1. Tableau aléatoire uniforme
   - A = np.random.rand(2, 3)
2. Statistiques
   - A.mean()
   - A.max()
3. prin(A)
4. B = np.random.randn(4, 4)
5. # Vérification statistique
6. print("Moyenne expérimentale :", B.mean())
7. print("Écart-type expérimental :", B.std())
8. randn(m, n) génère des valeurs suivant une distribution normale centrée réduite (courbe de Gauss).En pratique, les valeurs varient souvent entre -3 et +3.

# Fonctions universelles



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Fonctions universelles

- Une *fonction universelle* (le terme exact est « `ufunc` », abréviation de *universal function*) est une fonction qui peut s'appliquer terme à terme aux éléments d'un tableau.
- Si $f$ est une *ufunc* et si $a = [a_0, a_1, \ldots, a_{n-1}]$ est un tableau, alors $f(a)$ renvoie le tableau $[f(a_0), f(a_1), \ldots, f(a_{n-1})]$.
- Les $a_k$ peuvent être des tableaux, par exemple les lignes d'une matrice $m$. La fonction $f$ s'applique alors récursivement aux éléments de $m$.
- Un grand nombre de fonctions usuelles sont directement « universalisées » dans numpy.
- Les fonctions mathématiques gardent le même nom (préfixé par np si on a importé numpy par import numpy as np).
- Si on effectue une opération arithmétique entre un tableau $a$ et un scalaire $x$, tout se passe comme si $x$ était élevé au rang de tableau constant de même format que $a$.

# Fonctions universelles

```
>>> a = np.arange(1,10); a
array([1, 2, 3, 4, 5, 6, 7, 8, 9])

>>> a + 1
array([ 2,  3,  4,  5,  6,  7,  8,  9, 10])

>>> 10 * a
array([10, 20, 30, 40, 50, 60, 70, 80, 90])

>>> a + a
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18])

>>> a * a
array([ 1,  4,  9, 16, 25, 36, 49, 64, 81])
```

```
>>> m = np.arange(15).reshape(3,5); m
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
>>> m + 10
array([[10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])
>>> m * m  # ce n'est pas le produit matriciel!!
array([[  0,   1,   4,   9,  16],
       [ 25,  36,  49,  64,  81],
       [100, 121, 144, 169, 196]])
```

$$(m * m)[i, j] = m[i, j] \times m[i, j]$$

```
>>> a = np.arange(1,13); a
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
>>> 2 ** a       # 2 à la puissance chaque élément de a
array([ 2,  4,  8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096], dtype=int32)
>>> 1/a          # le vecteur des inverses
array([ 1.        ,  0.5       ,  0.33333333,  0.25      ,  0.2       ,  0.16666667,
        0.14285714,  0.125     ,  0.11111111,  0.1       ,  0.09090909,  0.08333333])
>>> np.log(a)    # le vecteur des logarithmes népériens
array([ 0.        ,  0.69314718,  1.09861229,  1.38629436,  1.60943791,  1.79175947,
        1.94591015,  2.07944154,  2.19722458,  2.30258509,  2.39789527,  2.48490665])
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Opérations arithmétiques

- `add(a,b)` additionne terme à terme les éléments de $a$ et $b$ (autre syntaxe possible : $a + b$)
- `subtract(a,b)` soustrait terme à terme les éléments de $b$ à ceux de $a$ (autre syntaxe possible : $a - b$)
- `multiply(a,b)` multiplie terme à terme les éléments de $a$ et $b$ (autre syntaxe possible : $a * b$)
- `divide(a,b)` quotients (flottants) terme à terme des éléments de $a$ par ceux de $b$ (autre syntaxe : $a/b$)

```
>>> a = np.arange(0,600,step=120);a
array([ 0, 120, 240, 360, 480])
>>> b  = np.array([10**k for k in range (5)]); b
array([ 1, 10, 100, 1000, 10000])
```

```
>>> np.divide(a,b)
array([ 0., 12., 2.4, 0.36, 0.048])
>>> a/b
array([ 0., 12., 2.4, 0.36, 0.048])
```

# Fonctions mathématiques usuelles

- Toutes les fonctions mathématiques usuelles sont présentes sous forme universelle dans le module numpy. Elles gardent le même nom, mais comme nous l'avons déjà indiqué, il faut bien penser à les préfixer par np.
- On dispose notamment des fonctions trigonométriques (circulaires et hyperboliques) directes et inverses :

```
sin cos tan arcsin arccos arctan sinh cosh tanh arcsinh arccosh arctanh
…
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Tests et comparaisons sur des tableaux



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Comparaisons entre tableaux

- `array_equal(a,b)` répond `True` si les tableaux $a$ et $b$ ont même format et même contenu, et `False` sinon.

```
>>> a = np.arange(0,12).reshape(3,4); a
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
>>> b = np.copy(a)
>>> np.array_equal(a,b)  # mêmes tableaux
True
```

```
>>> b = b.reshape((4,3)); b # mêmes élts, forme ≠
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
>>> np.array_equal(a,b)     # donc tableaux ≠
False
```

# Comparaisons entre tableaux

- `greater(a,b)` renvoie le tableau des booléens $a_k > b_k$, où $a_k$ et $b_k$ se correspondent dans $a$ et $b$.

```
>>> a = np.random.rand(7); a
array([ 0.45268328,  0.89699802,  0.47090844,  0.06040679,  0.92796843,  0.69654813,  0.64937467])
>>> b = np.random.rand(7); b
array([ 0.73789601,  0.70574489,  0.1792136 ,  0.48761371,  0.91307424,  0.54268789,  0.81540996])
>>> np.greater(a,b)    # pour chaque k, teste si a[k] > b[k]
array([False,  True,  True, False,  True,  True, False], dtype=bool)
```

- On a des possibilités analogues avec : `greater_equal(a,b)` (test $a_k \geq b_k$), `less(a,b)` (test $a_k < b_k$), `less_equal(a,b)` (test $a_k \leq b_k$), `equal(a,b)` (test $a_k = b_k$) et `not_equal(a,b)` (test $a_k \neq b_k$) .
- Les scalaires sont étendus en des tableaux constants pour permettre les comparaisons terme à terme :

```
>>> a = np.random.random_integers(100,1000,10); a
array([866, 305, 892, 849, 763, 433, 322, 442, 849, 239])
>>> np.less(a,500)              # teste si les a[k] vérifient a[k]<500
array([False,  True, False, False, False,  True,  True,  True, False,  True], dtype=bool)
>>> a = np.random.random_integers(10,size=12); a
array([ 6,  9,  4,  5,  5,  2,  3,  7,  9,  8,  4])
>>> np.equal(a,5)              # teste si les éléments sont égaux  à 5
array([False, False, False,  True,  True, False, False, False, False, False, False, dtype=bool)
```

# Tris de tableaux

- L'expression `sort(a)` renvoie une copie triée du tableau *a* (l'original n'est donc pas modifié).
- Mais attention : l'expression `a.sort()` trie le tableau *a* sur place (il y a donc une différence importante entre la fonction `sort` du module numpy et la méthode `sort` d'un objet numpy particulier).

```
>>> a = np.random.randint(100,1000,size=13); a
array([514, 553, 400, 234, 449, 887, 240, 252, 317, 231, 522, 578, 548])
>>> np.sort(a)              # on utilise la fonction sort du module numpy.
array([231, 234, 240, 252, 317, 400, 449, 514, 522, 548, 553, 578, 887])
>>> a                       # le tableau a original n'est donc pas été modifié
array([514, 553, 400, 234, 449, 887, 240, 252, 317, 231, 522, 578, 548])
>>> a.sort()                # on utilise la méthode sort de l'objet
>>> a                       # cette fois-ci le tri a été effecuté sur place
array([231, 234, 240, 252, 317, 400, 449, 514, 522, 548, 553, 578, 887])
```

# Tris de tableaux

- L'expression `argsort(a)` renvoie le tableau des indices i tel que `ai[k]` est la $k$-ème plus petite valeur de $a$ :

```
>>> a = np.random.randint(100,1000,size=13); a
array([357, 136, 784, 999, 965, 172, 159, 935, 330, 609, 128, 518, 986])
>>> i = np.argsort(a); i     # le tableau des indices des éléments de a dans l'ordre croissant
array([10,  1,  6,  5,  8,  0, 11,  9,  2,  7,  4, 12,  3], dtype=int32)
>>> a[i[0]]                  # le plus petit élément de a
128
>>> a[i[5]]                  # le cinquième plus petit élément de a
357
>>> a[i]                     # permet de retrouver une copie triée de a
array([128, 136, 159, 172, 330, 357, 518, 609, 784, 935, 965, 986, 999])
```

# Minimum et maximum

- `amin(a[,axis,out,keepdims])` renvoie le(s) minimum(s) d'un tableau, éventuellement selon un « axe » (une direction)

```
>>> a = np.random.random_integers(100,1000, size=(5,6)); a
array([[690, 266, 954, 806, 903, 473],
       [536, 351, 939, 331, 552, 162],
       [745, 981, 423, 224, 927, 840],
       [149, 849, 884, 961, 158, 494],
       [354, 318, 604, 214, 124, 253]])
```

```
>>> np.amin(a)                    # le minimum de a aplati
124
>>> np.amin(a,0)                  # suivant le premier indice
array([149, 266, 423, 214, 124, 162])
>>> np.amin(a,1)
array([266, 162, 224, 149, 124]) # suivant le deuxième indice
```

```
>>> np.amin(a,1,keepdims=True)
array([[266],
       [162],
       [224],
       [149],
       [124]])
```

- `amax(a[,axis,out,keepdims])` est l'analogue de `amin`, mais pour la recherche de maximum.

# Minimum et maximum

- `argmax(a[,axis])` et `argmin(a[,axis])` renvoie les indices où se trouvent le ou les éléments maximum(s) (respectivement minimum(s)) dans un tableau. On reprend à nouveau le tableau *a* de l'exemple précédent.

```
>>> np.argmin(a), np.argmax(a)          # la position du min (resp du max) dans le tableau à plat
(28, 13)
>>> np.argmin(a,0)                      # n° de ligne des éléments minimuns, colonne par colonne
array([3, 0, 2, 4, 4, 1], dtype=int32)
>>> np.argmax(a,0)                      # n° de ligne des éléments maximums, colonne par colonne
array([2, 2, 0, 3, 2, 2], dtype=int32)
>>> np.argmin(a,1)                      # n° de colonne des éléments minimuns, ligne par ligne
array([1, 5, 3, 0, 4], dtype=int32)
```

# Recherches dans un tableau

- `any(a)` teste si le tableau *a* contient au moins un élément « vrai » (au sens booléen ou scalaire non nul).

```
>>> a = np.random.random_integers(10,99,size=20); a
array([19, 15, 34, 54, 76, 16, 75, 30, 90, 21, 25, 49, 47, 57, 94, 52, 39, 34, 21, 15])
>>> np.any(a>90)
True
>>> a = np.random.random_integers(10,99,size=(5,6)); a
array([[64, 13, 69, 42, 44, 46],
       [72, 97, 82, 27, 27, 34],
       [75, 14, 36, 11, 16, 86],
       [83, 23, 51, 57, 41, 38],
       [21, 97, 92, 85, 12, 41]])
>>> np.any(a>90,axis=0)   # colonne par colonne, teste s'il y a au moins un élément > 90
array([False,  True,  True, False, False, False], dtype=bool)
>>> np.any(a>90,axis=1)   # ligne par ligne, teste s'il y a au moins un élément > 90
array([False,  True, False, False,  True], dtype=bool)
```

# Recherches dans un tableau

- `all(a)` teste si tous les éléments du tableau *a* sont « vrais » (au sens booléen ou scalaire non nul).

```
>>> a = np.random.random_integers(10,99,size=20); a
array([68, 50, 46, 51, 86, 12, 42, 45, 74, 36, 64, 88, 98, 87, 63, 18, 81, 99, 90, 54])
>>> np.all(a>10)   # est-ce que tous les éléments de a sont > 10 ?
True
>>> a = np.random.random_integers(10,99,size=(3,10)); a
array([[72, 37, 46, 28, 64, 30, 40, 22, 58, 46],
       [77, 74, 98, 21, 28, 90, 53, 56, 66, 38],
       [49, 75, 54, 28, 55, 90, 74, 68, 81, 71]])
>>> np.all(a % 2 == 0, axis=0)  # colonne par colonne, est-ce que tous les éléments sont pairs?
array([False, False,  True, False, False,  True, False,  True, False, False], dtype=bool)
>>> np.all(a % 17, axis=1)   # ligne par ligne, tous les éléments sont-ils non divisibles par 17 ?
array([ True,  True, False], dtype=bool)
```

# Recherches dans un tableau

- `where(condition)` renvoie le tableau des positions des éléments d'un vecteur qui possèdent une propriété particulière.

```
>>> a = np.random.random_integers(100,1000,15); a
array([318, 789, 611, 310, 254, 453, 864, 155, 400, 342, 377, 117, 796, 297, 949])
>>> i = np.where(a % 2); i      # tableau des positions des éléments impairs
(array([ 1,  2,  5,  7, 10, 11, 13, 14], dtype=int32),)
>>> a[i]                        # récupère le tableau de ces éléments impairs
array([789, 611, 453, 155, 377, 117, 297, 949])
```

- La fonction where admet deux arguments facultatifs sous la forme d'un tableau *a* et d'un tableau *b*. À chaque fois que la *condition* qui est le premier argument de where est vraie, c'est l'élément correspondant de *a* qui est renvoyé, sinon c'est l'élément de *b*. Tout ça demande un peu d'habitude mais peut conduire à des constructions assez élégantes.

# Tableaux d'un point de vue ensembliste

- L'expression `unique(a)` renvoie les éléments de *a* triés, avec suppression des doublons :

```
>>> a = np.array([5,8,3,2,4,2,3,5,7])
>>> np.unique(a)
array([2,3,4,5,7,8])
>>> np.unique(a,True)       # ajoute la position des éléments uniques sélectionnés
(array([2,3,4,5,7,8]), array([3,2,4,0,8,1], dtype=int32))
>>> np.unique(a,True,True)  # renvoie en plus les indices pour reconstruire le tableau initial
(array([2,3,4,5,7,8]), array([3,2,4,0,8,1], dtype=int32), array([3,5,1,0,2,0,1,3,4], dtype=int32))
```

# Sommes, produits, différences

- L'expression `a.sum()` renvoie la somme des éléments de *a*.
- On dispose aussi de a.cumsum() (sommes cumulées), `a.prod()` (produit) et `a.cumprod()` (produits cumulés).
- On dispose comme souvent de deux syntaxes possibles : utiliser la méthode du tableau *a* (par exemple `a.sum()`) ou une fonction du module numpy appliquée au tableau *a* (par exemple `np.sum(a)`).

```
>>> a = np.array([5,8,3,2,6]); a
array([5, 8, 3, 2, 6])
>>> np.sum(a)
24
>>> a.sum()
24
```

```
>>> np.cumsum(a)
array([ 5, 13, 16, 18, 24], dtype=int32)
>>> a.prod()
1440
>>> a.cumprod()
array([ 5, 40, 120, 240, 1440], dtype=int32)
```

# Calculs statistiques

- On dispose des fonctions `mean` (moyenne arithmétique), `var` (variance) et `std` (écart-type)

```
>>> a.mean()          # moyenne arithmétique
4.7999999999999998
>>> a.var()           # variance
4.5600000000000005
```

```
>>> a.std()           # écart-type
2.1354156504062622
>>> a.std()**2        # (écart-type)² => variance
4.5599999999999996
```

- Toutes ces fonctions s'appliquent à une matrice. On peut alors préciser un axe de calcul (`axis=0` pour un calcul sur les lignes et `axis=1` pour un calcul sur les colonnes). Si on ne précise pas cet argument, le calcul est fait « à plat ».

```
>>> a = np.arange(20).reshape(4,5)
>>> a
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

```
>>> a.sum()          # somme du tableau « aplati »
190
>>> a.sum(axis=0)    # somme des lignes
array([30, 34, 38, 42, 46])
>>> a.sum(axis=1)    # somme des colonnes
array([10, 35, 60, 85])
```

- On dispose également des fonctions `cov` (matrice de covariance), et `corrcoef` (coefficients de corrélation).

# Calcul matriciel



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Opérations linéaires

- Les combinaisons linéaires de vecteurs ou de matrices s'effectuent de façon très naturelle :

```
>>> x = 100
>>> y = -1
>>> a = np.arange(6); a
array([0, 1, 2, 3, 4, 5])
```

```
>>> b = np.arange(1,13,2); b
array([ 1,  3,  5,  7,  9, 11])
>>> x*a+y*b
array([ -1,  97, 195, 293, 391, 489])
```

# Produits matriciels

- Le **produit matriciel** est une opération entre **deux matrices** (ou un vecteur et une matrice) qui consiste à **combiner les lignes de la première matrice** avec **les colonnes de la seconde**.
- Chaque élément du résultat est le **produit scalaire** entre **une ligne de la première matrice** et **une colonne de la seconde**.
- Le produit matriciel s'en déduit : le produit de la matrice $A\ (m \times n)$ par la matrice $B\ (n \times p)$ est la matrice $C\ (m \times p)$ telle que l'élément $C_{ij}$ est égal au produit scalaire de la ligne $i$ de la matrice $A$ par la colonne j de la matrice $B$.

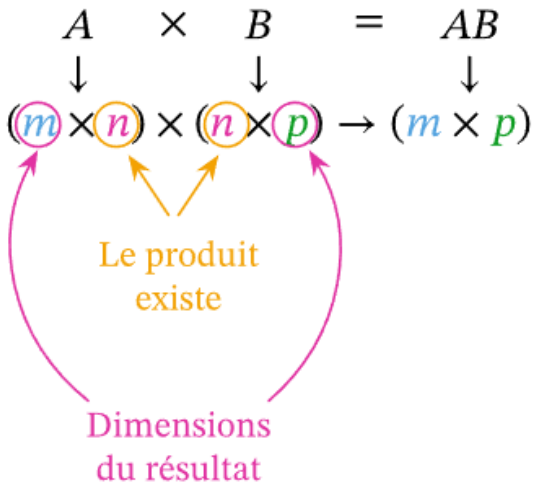
**Exemple :**

A est de taille (2×3)

B est de taille (3×4)

On peut calculer A × B,

et le **résultat aura la taille (2×4)**.



# Produits matriciels

## Règle de calcul

- SI

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \quad \text{et} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{bmatrix}$$

- Alors

$$A \times B = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} & a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} & a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32} \end{bmatrix}$$

- Chaque case du résultat est donc : une **somme de produits** : ligne de A × colonne de B.


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

# Produits matriciels

- `dot(a,b[,out])` effectue le produit matriciel *ab*, et (facultativement) place le résultat dans *out*.
- Pour deux vecteurs `a = [a1,...,an]` et `b = [b1,...,bn]`, on calcule ici le produit scalaire $\sum_{k=1}^{n} a_k b_k$.
- Un vecteur *v* est considéré comme une ligne à gauche, et une colonne à droite.

```
>>> a = np.arange(1,5); a
array([1, 2, 3, 4])
>>> b = np.arange(11,15); b
array([11, 12, 13, 14])
>>> c = np.arange(21,26); c
array([21, 22, 23, 24, 25])
>>> m = np.arange(20).reshape(4,5); m
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

```
>>> np.dot(a,b)   # produit scalaire a.b
130
>>> np.dot(a,m)   # vecteur-ligne par matrice
array([100, 110, 120, 130, 140])
>>> np.dot(m,c)   # matrice par vecteur-colonne
array([ 240,  815, 1390, 1965])
>>> np.dot(m,m.T) # calcule m.m^t
array([[  30,   80,  130,  180],
       [  80,  255,  430,  605],
       [ 130,  430,  730, 1030],
       [ 180,  605, 1030, 1455]])
```

# Exercice

- Créer deux tableaux NumPy A et B :

```
A = np.array([2, 4, 6, 8])
B = np.array([1, 3, 5, 7])
```

- Effectuer à l'aide des fonctions universelles :la somme de A et B, la différence A - B, le produit A * B, la division A / B, la puissance $A^2$, le reste de la division A % B.
- Crée un tableau `B = np.array([3, 7, 1, 9, 4, 2, 8, 0, 5, 6]).`À l'aide de comparaisons, affiche :
  - les éléments de B supérieurs à 5,
  - un tableau booléen indiquant les positions où B est pair,
- Crée une matrice M1 de taille (2×3) et une autre matrice M2 de taille (3×2) avec np.array().
  - Calcule :le produit matriciel M1 × M2,le produit M2 × M1,et le produit élément par élément (Hadamard) de deux matrices de même taille